Engaging GPU Howto Recipes PyTorch

Intermediate Distributed Deep Learning with PyTorch

Deep learning is the foundation of artificial intelligence nowadays. Deep learning programs can be accelerated substantially on GPUs.

There are various parallelisms to enable distributed deep learning on multiple GPUs, including data parallel and model parallel.

We have introduced basic recipes of data parallel with PyTorch, which is a popular Python package for working on deep learning projects.

In data parallel, the model has to fit into the GPU memory. However, large model sizes are required for large language models (LLMs) based on the transformer architecture. When the model does not fit into the memory of a single GPU, the normal data parallelism does not work.

On this page, we will introduce intermediate recipes to train large models on multiple GPUs with PyTorch.

First, there is a Fully Sharded Data Parallel (FSDP) approach to split the model into multiple GPUs so that the memory requirement fits. A shard of the model is stored in each GPU, and communication between GPUs happens during the training process. We will introduce FSDP recipes in the first section.

However, FSDP does not gain additional speedup beyond the data parallel framework. Better approaches are based on model parallel, which not only splits the model into multiple GPUs but also accelerates the training process with parallel sharded computations. There are various schemes of model parallel, such as pipeline parallel (PP) and tensor parallel (TP). Usually, model parallel is applied on top of data parallel to gain further speedup. In the second section, we will focus on recipes of hybrid Fully Sharded Data Parallel and Tensor Parallel (referred to as FSDP + TP) .

Installing PyTorch

First, load a Miniforge module to provide a Python platform with PyTorch and CUDA support preinstalled,

```
module load miniforge/24.3.0-0
```

Fully Sharded Data Parallel

We use an example code to train a convolutional neural network (CNN) with the MNIST data set.

We will first run the example on a single GPU and then extend it to multiple GPUs with FSDP.

Download the codes mnist_gpu.py and FSDP_mnist.py for these two cases respectively.

An example with a single GPU

To run the example on a single GPU, prepare a job script named `job.sh` like this,

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH --job-name=mnist-gpu
#SBATCH -N 1
#SBATCH -n 1
#SBATCH --mem=20GB
#SBATCH --gres=gpu:h200:1

module load miniforge/24.3.0-0

python mnist_gpu.py
```

Here we sepecify the GPU type of H200 with `--gres=gpu:h200:1`. If one is not particular about the type of GPU, `--gres=gpu:1` can be used instead.

Submit the job script,

```
sbatch job.sh
```

While the job is running, you can check if the program runs on a GPU. First, check the hostname that it runs on,

```
squeue --me
```

and then log in to the node,

```
ssh <nodeXXX>
```

and check the GPU usage with the `nvtop` command.

Single-node multi-GPU FSDP

Now we extend this example to multiple GPUs on a single node with FSDP.

Prepare a job script named `job.sh` like this,

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH --job-name=fsdp
#SBATCH -N 1
#SBATCH -n 2
#SBATCH --mem=20GB
#SBATCH --gres=gpu:h200:2

module load miniforge/24.3.0-0

python FSDP_mnist.py
```

then submit it,

```
sbatch job.sh
```

As set up in the program `FSDP_mnist.py`, it will run on all GPUs requested in Slurm, that is 2 in this case. That says the model is split into 2 shards, each stored on a GPU, and the training process happens on 2 batches of data simultaneously. Communication between GPUs happens under the hood.

Hybrid Fully Sharded Data Parallel and Tensor Parallel

Tensor parallel can be applied on top of data parallel to gain further speedup. In this section, we introduce recipes of hybrid FSDP and TP.

We use an example that implements FSDP + TP on LLAMA2 (Large Language Model Meta AI 2). Refer to the description of this example. Download the codes: fsdp_tp_example.py, llama2_model.py, and log_utils.py.

Single-node multi-GPU FSDP + TP

First, let's run the example on multiple GPUs within a single node.

The code `fsdp_tp_example.py` is set up for this purpose. The TP size is set to be 2 in the code. The total number of GPUs should be equal to a multiple of the TP size, then the FSDP size is equal to the number of GPUs divided by the TP size.

To run this example on multiple GPUs, prepare a job script like this,

```
#!/bin/bash
#SBATCH -p mit_preemptable
#SBATCH -t 60
#SBATCH -N 1
#SBATCH -n 4
#SBATCH --mem=30GB
#SBATCH --gres=gpu:h200:4

module load miniforge/24.3.0-0

torchrun --nnodes=1 --nproc_per_node=4 \
     --rdzv_id=$SLURM_JOB_ID \
     --rdzv_endpoint="localhost:1234" \
     fsdp_tp_example.py
```

then submit it,

```
sbatch job.sh
```

With the flags `--nnodes=1 --nproc-per-node=4`, the `torchrun` command will run the program on 4 GPUs within one node. The training process happens on 2 batches of data with FSDP, and the model is trained with TP sharded computation on 2 GPUs for each batch of data.

The flags with `rdzv` (meaning the Rendezvous protocol) are required by `torchrun` to coordinate multiple processes. The flag `--rdzv-id=$SLURM_JOB_ID` sets to the `rdzv` ID to be the job ID, but it can be any random number. The flag `--rdzv-endpoint=localhost:1234` is to set the host and the port. Use `localhost` when there is only one node. The port can be any 4- or 5-digit number larger than 1024.

Multi-node multi-GPU FSDP + TP

Finally, we run this example on multiple GPUs across multiple nodes. Note that this example requires 8 GPUs which is more than the standard user has access to on Engaging. If you have access to more GPUs, run this example on the corresponding partition.

Prepare a job script like this,

```
#!/bin/bash
#SBATCH -p  mit_normal_gpu
```

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=4
#SBATCH --gpus-per-node=h200:4
#SBATCH --mem=30GB

module load miniforge/24.3.0-0

# Get IP address of the master node
nodes=( $( scontrol show hostnames $SLURM_JOB_NODELIST ) )
nodes_array=($nodes)
master_node=${nodes_array[0]}
master_node_ip=$(srun --nodes=1 --ntasks=1 -w "$master_node" hostname --ip-
address)

srun torchrun --nnodes=$SLURM_NNODES \
          --nproc-per-node=$SLURM_CPUS_PER_TASK \
          --rdzv-id=$SLURM_JOB_ID   \
          --rdzv-backend=c10d \
          --rdzv-endpoint=$master_node_ip:1234 \
          fsdp_tp_example.py
```

then submit it,

```
sbatch job.sh
```

The configuration of the `#SBATCH` and `torchrun` flags is similar to that in the basic recipe of data parallel.

The program runs on 8 GPUs with 4 per node. As is set up in the code `fsdp_tp_example.py`, the training process happens on 4 batches of data with FSDP, and the model is trained with TP sharded computation on 2 GPUs for each batch of data.

Topology of GPU Communication

The NVIDIA Collective Communications Library (NCCL) is set as the backend in all of the PyTorch programs here, so that the communication between GPUs within one node benefits from the high bandwidth of NVLinks, and the communication between GPUs across nodes benefits from the bandwidth of the Infiniband network.

The intra-node GPU-GPU communication speed is much faster than the inter-node. The communication overhead of TP is much larger than that of FSDP. The topology of GPU communication is set up (in the code `fsdp_tp_example.py`) in a way that TP communication is intra-node and FSDP communication is inter-node, so that the usage of network bandwidth is optimized.

February 11, 2026